from having to duplicate code for the header and footer in every page, and mean that if you need to tweak something, you'll need to do it in only one place.

NOTE I cover Razor templates, including layouts, in detail in chapter 17. You can find layouts in the Pages/Shared folder of your project.

As you've already seen, you can include C# code in your Razor Pages by using curly braces `@{ }`, but generally speaking, you'll want to limit the code in your .cshtml file to presentational concerns only. Complex logic, code to access services such as a database, and data manipulation should be handled in the `PageModel` instead.

13.1.4 Handling request logic with page models and handlers

As you've already seen, the `@page` directive in a .cshtml file marks the page as a Razor Page, but most Razor Pages also have an associated *page model*. By convention, this page model is placed in a file commonly known as a *code-behind file* that has a .cs extension, as you saw in figure 13.5. Page models should derive from the `PageModel` base class, and they typically contain one or more methods called *page handlers* that define how to handle requests to the Razor Page.

DEFINITION A *page handler* is the Razor Pages equivalent of a minimal API endpoint handler; it's a method that runs in response to a request. Razor Page models must be derived from the `PageModel` class. They can contain multiple page handlers, though typically they contain only one or two.